



DIGITAL DESIGN

LAB3 DATA FLOW & TESTBENCH, GATES IN RTL VS LUT IN FPGA

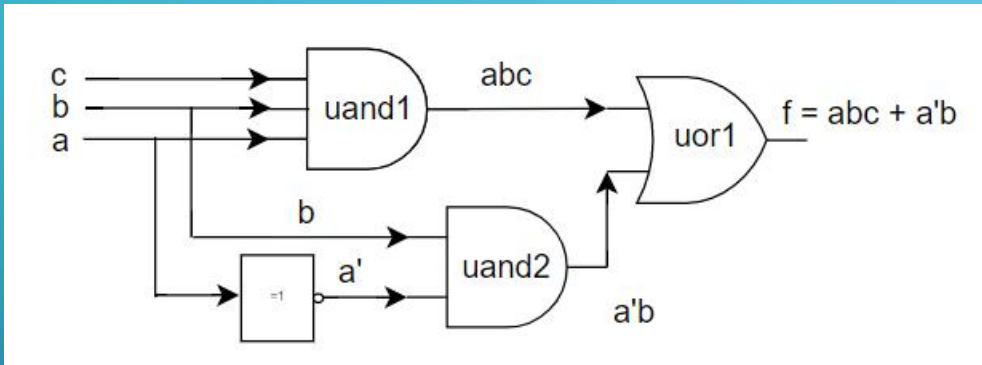
WANGW6@SUSTECH.EDU.CN

LAB3

- Design in Verilog
 - **Structrue Design**(review)
 - **Data Flow, assign, Bitwise and logic operations**
 - **Data Flow vs Structrue Design**
- Testbench in Verilog
 - System tasks and funcions
 - Loop (forever, for, while, repeat)
- Vivado Process
 - Schematic of “RTL analysis” (Gates in RTL)
 - Schematic of “Synthesis” (LUT in FPGA)
 - active module (set as top)

DESIGN MODE 1 IN VERILOG: STRUCTURE DESIGN BY USING THE PRIMITIVE GATES(REVIEW)

Do the design to impliment the following circuit:
 $f(a,b,c) = abc + a'b$



//describ the structure of the circuit in Verilog

```
module lab3_1_gates(
```

```
    input a,
```

```
    input b,
```

```
    input c,
```

```
    output f
```

```
);
```

```
//f(a,b,c) = abc + a'b
```

```
wire not_a, and1_or1, and2_or1;
```

```
and uand1(and1_or1, a, b, c);    //and1_or1 = abc
```

```
not unot1(not_a, a);            // nota = a'
```

```
and uand2(and2_or1, not_a, b);  //and2_or2 = a'b
```

```
or uor1 ( f, and1_or1, and2_or1); //f = abc + a'b
```

```
endmodule
```

DESIGN MODE 2 IN VERILOG: DATA FLOW

- **Data flow** : using “assign” as *continuous assignment* , to transfer the data from input ports through variables to the output ports .

logical expression	data flow in Verilog
$f(sw) = sw$	assign f = sw;
$f(a,b,c) = abc + a'b$	assign f = a & b & c ~a & b;
$f(a,b,c) = a'bc' + ab'c' + ab'c + abc'$	assign f = ~a&b&~c a&~b&~c a& ~b&c a&b&~c;

TIPS:

The priority of operator “ & ” is higher than the operator “ | ”

BITWISE AND LOGICAL OPERATIONS IN VERILOG(1)

Four-valued logic (The IEEE 1364 standard): 0, 1, Z (high impedance), and X (unknown logic value).

Operator :

~ & ^ ~^ ^~ | ! && ||

Priority (from high to low) :

~ ! >

& >

^ ~^ ^~ >

| >

&& >

||

Operator type	Operator symbols	Operation performed
Bitwise	~	Bitwise NOT (1's complement)
	&	Bitwise AND
		Bitwise OR
	^	Bitwise XOR
	~^ or ^~	Bitwise XNOR
Logical	!	NOT
	&&	AND
		OR

BITWISE AND LOGICAL OPERATIONS IN VERILOG(2)

Tips:

While the **bit-width of the operand is 1**, the bitwise operation is **same** as the corresponding logical operation.

While the **bit-width of the operand is more than 1**, the bitwise operation is **NOT** always **same** as the corresponding logical operation.

a	b	a b	a b	a & b	a && b	~a	!a
2'b01	2'b10	2'b11	2'b01	2'b00	2'b01	2'b10	2'b00
2'b11	2'b11	2'b11	2'b01	2'b11	2'b01	2'b00	2'b00
2'b00	2'b10	2'b10	2'b01	2'b00	2'b00	2'b11	2'b01
...	...	...	...	...	...	...	...

The relationship between boolean and number in Verilog:

- 1) Zero is taken as **False**, None Zero is taken as **True**
- 2) **False** is represented by zero, **True** is represented by one.

DATA FLOW VS STRUCTURE DESIGN(BASED ON THE PRIMITIVE GATES)

Data Flow in Verilog

logical expression: $f(a,b,c) = abc + a'b$

```
assign f = a & b & c | ~a & b;
```

```
wire and_abc, and_na_b;
```

```
assign and_abc = a & b & c;
```

```
assign and_na_b = ~a & b;
```

```
assign f = and_abc | and_na_b;
```

TIPS:

Both 1 *continuous* assignment statement or several *continuous* assignment statements are ok.

Structure Design in Verilog (Based on the primitive Gates)

logical expression: $f(a,b,c) = abc + a'b$

```
wire not_a, and1_or1, and2_or1;
```

```
and uand1(and1_or1, a, b, c);
```

```
not unot1(not_a, a);
```

```
and uand2(and2_or1, not_a, b);
```

```
or uor1(f, and1_or1, and2_or1);
```

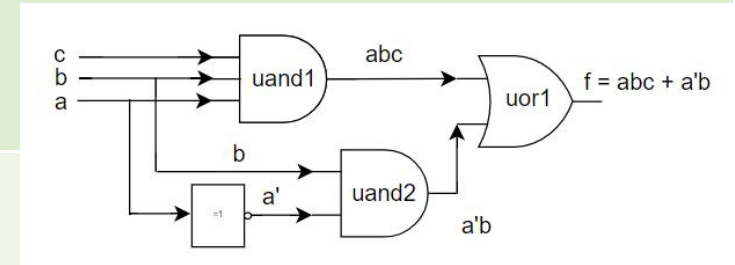
```
wire not_a, and1_or1, and2_or1;
```

```
or uor1(f, and1_or1, and2_or1);
```

```
not unot1(not_a, a);
```

```
and uand2(and2_or1, not_a, b);
```

```
and uand1(and1_or1, a, b, c);
```



TIPS: The **order** in which statements **not in the range of 'begin' and 'end'** are written does not affect the description of the circuit, as **Verilog** has the **parallelism** characteristic.

DESIGN, TESTBENCH AND WAVEFORM

Demo: a) $q1 = x$ b) $q2 = x + xy$ c) $q3 = x(x + y)$

circuit design,
Design source file

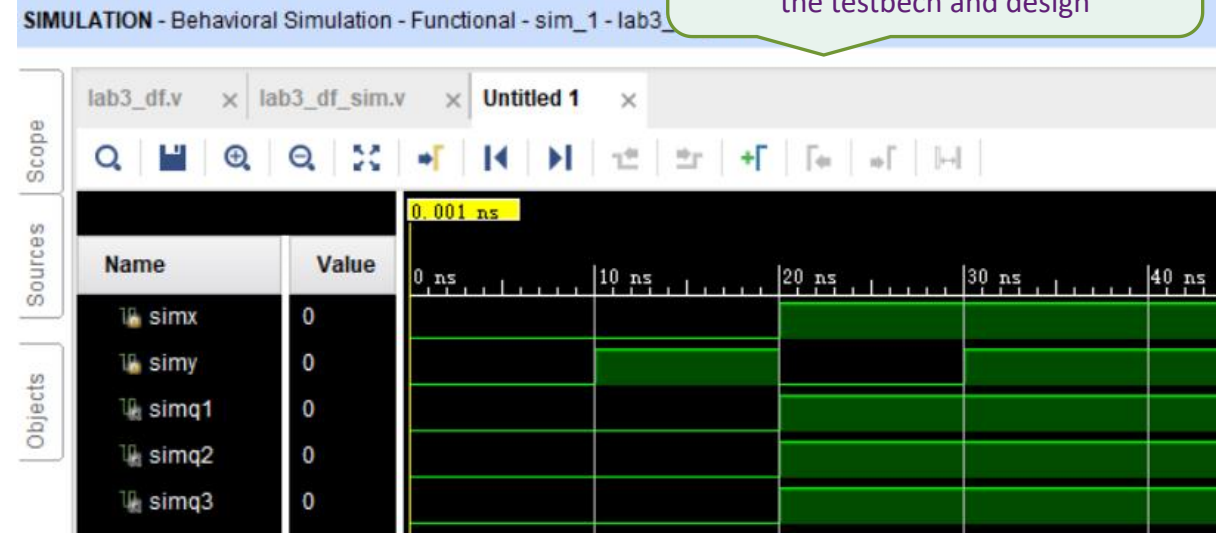
```
module lab3_df(  
    input x,  
    input y,  
    output q1,  
    output q2,  
    output q3  
);  
    assign q1 = x;  
    assign q2 = x | (x & y);  
    assign q3 = x & (x | y);  
endmodule
```

```
module lab3_df_sim( );  
    reg simx, simy;  
    wire simq1, simq2, simq3;  
    lab3_df u_df(  
        .x(simx), .y(simy), .q1(simq1), .q2(simq2), .q3(simq3) );  
  
    initial  
    begin  
        simx=0;  
        simy=0;  
        #10  
        simx=0;  
        simy=1;  
        #10  
        simx=1;  
        simy=0;  
        #10  
        simx=1;  
        simy=1;  
    end  
endmodule
```

Testbench,
Simulation source file

waveform

generated by the simulator based on
the testbench and design



Q1. How long the simulation will take?
Q2. Is there any other way to show
the simulation result ?

SYSTEM TASKS AND FUNCTIONS IN VERILOG(1)

System tasks and functions: The **\$** character introduces a language construct that enables development of user-defined tasks and functions. System constructs are not design semantics, but refer to **simulator** functionality. A name following the \$ is interpreted as a system task or a system function.

- Text Output System Tasks: **\$display**, \$write, **\$monitor**, \$strobe, etc.
- File I/O System Tasks and Functions: \$fopen, \$fclose, \$fdisplay, \$fmonitor, etc.
- Other Common System Tasks and Functions: **\$finish**, \$stop, **\$time**, \$realtime, etc.

Tip: the System tasks and functions in Verilog are commonly used for testbench files, Not for design files.

SYSTEM TASKS AND FUNCTIONS IN VERILOG(2)

➤ `$display("text_with_format_specifiers", list_of_arguments);`

- Prints the formatted message when the statement is executed. A newline is automatically added to the message.

➤ `$monitor("text_with_format_specifiers", list_of_arguments);`

- Invokes a background process that continuously monitors the arguments listed, and prints the formatted message whenever one of the arguments changes. A newline is automatically added to the message.

Text Formatting Codes			
%b	binary values	%m	hierarchical name of scope
%o	octal values	%l	configuration library binding
%d	decimal values	\t	print a tab
%h	hex values	\n	print a newline
%e	real values-exponential	\"	print a quote
%f	real values-decimal	\\	print a backslash
%t	formatted time values	%%	print a percent sign
%s	character strings		

%Ob, %Oo, %Od and %Oh truncates any leading zeros in the value.
%e and %f may specify field widths (e.g. %5.2f).
%m and %l do not take an argument; they have an implied argument value.
The format letters are not case sensitive (i.e. %b and %B are equivalent).

SYSTEM TASKS AND FUNCTIONS IN TESTBENCH

```
module cs207_lab3_tb( );

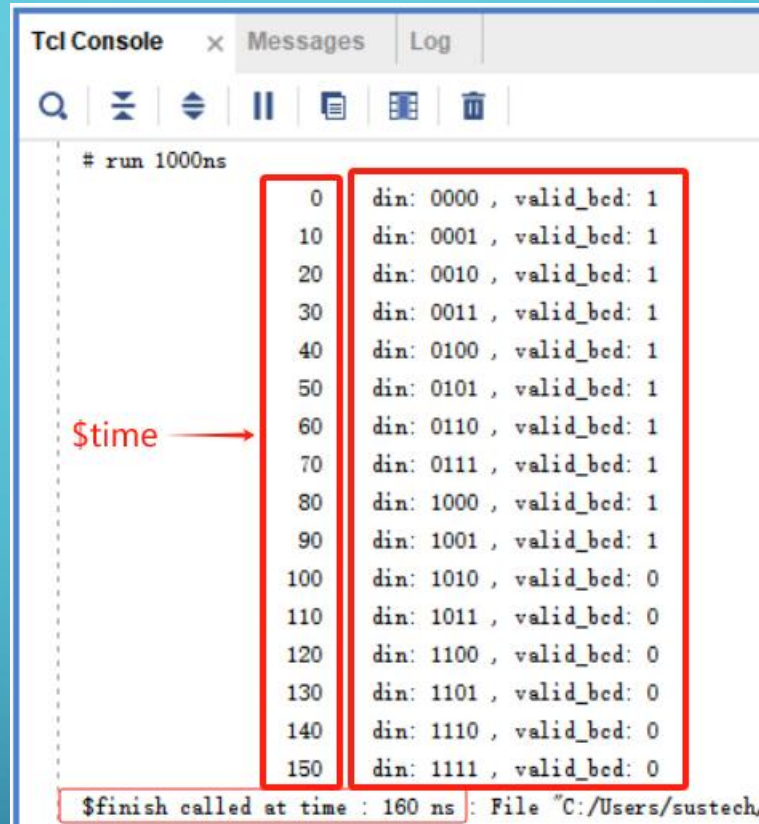
reg [3:0] din=4'b0000;
wire valid_bcd;
cs207_lab3_bcd dut(din, valid_bcd);

always #10 din = din+1;

initial
$monitor($time,
"\tdin: %4b , valid_bcd: %1b",
din, valid_bcd);

initial #160 $finish;

endmodule
```



```
Tcl Console x Messages Log
# run 1000ns
0    din: 0000 , valid_bcd: 1
10   din: 0001 , valid_bcd: 1
20   din: 0010 , valid_bcd: 1
30   din: 0011 , valid_bcd: 1
40   din: 0100 , valid_bcd: 1
50   din: 0101 , valid_bcd: 1
60   din: 0110 , valid_bcd: 1
70   din: 0111 , valid_bcd: 1
80   din: 1000 , valid_bcd: 1
90   din: 1001 , valid_bcd: 1
100  din: 1010 , valid_bcd: 0
110  din: 1011 , valid_bcd: 0
120  din: 1100 , valid_bcd: 0
130  din: 1101 , valid_bcd: 0
140  din: 1110 , valid_bcd: 0
150  din: 1111 , valid_bcd: 0
$finish called at time : 160 ns : File "C:/Users/sustech/
```

TIPS:

In Vivado, the formatted message printed by system tasks and functions would be displayed in the “**Tcl Console**” sub-window.



LOOP IN TESTBENCH(1)

The function of { } in Verilog is to concatenate multiple signals in a certain order into a temporary object.

```
1  timescale 1ns / 1ps
2
3  module sop_som_pom_sim( );
4  reg sa, sb, sc;
5  wire sz1, sz2, sz3;
6  sop_som_pom u1( .a(sa), .b(sb), .c(sc), .z1(sz1), .z2(sz2), .z3(sz3) );
7
8  initial begin
9  {sa, sb, sc} = 3'b0;
10 forever #100 {sa, sb, sc} = {sa, sb, sc} + 1;
11 end
12
13 initial #800 $finish;
14
15 endmodule
```



Tips on Vivado:

- Select the name of the ports you want to see in group (ctrl + select)
- Right click then choose the “New virtual Bus” to group all the selected ports, the name of the virtual bus could be edit

LOOP IN TESTBENCH(2)

```
initial begin
    {sa, sb, sc} = 3'b0;
    repeat(7) begin
        #100 {sa, sb, sc} = {sa, sb, sc} + 1;
    end
    #100 $finish();
end
```

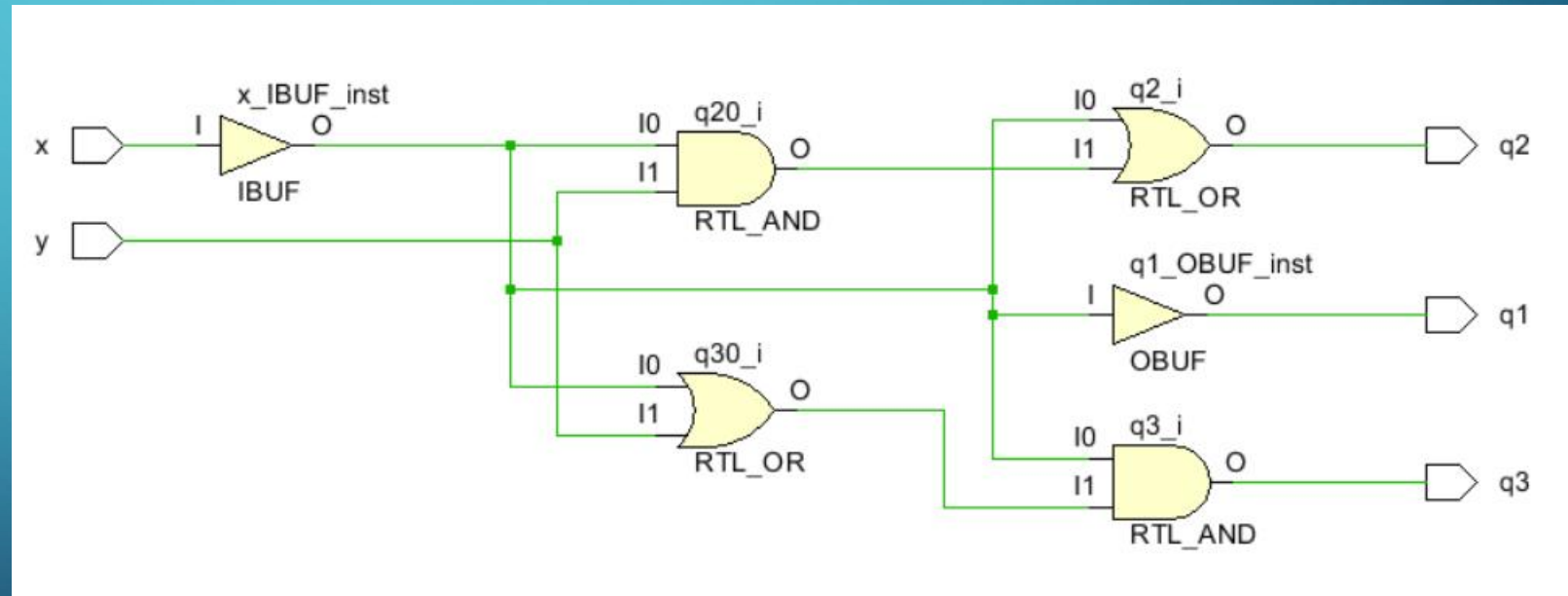
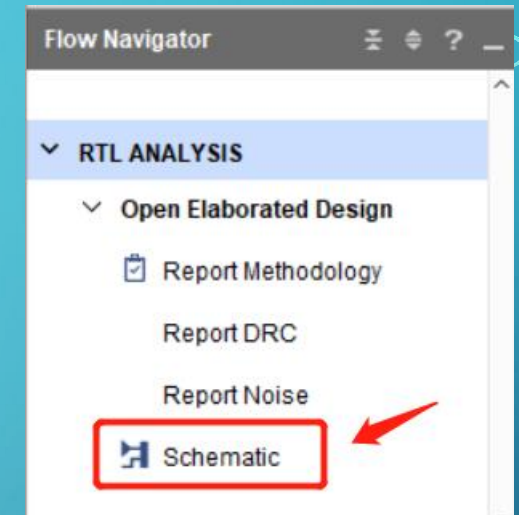
```
initial begin
    {sa, sb, sc} = 3'b0;
    for(integer i=0; i<7; i=i+1) begin
        #100 {sa, sb, sc} = {sa, sb, sc} + 1;
    end
    #100 $finish();
end
```

```
integer i=0;
initial begin
    {sa, sb, sc} = 3'b0;
    while(i<7) begin
        #100 {sa, sb, sc} = {sa, sb, sc} + 1;
        i=i+1;
    end
    #100 $finish();
end
```

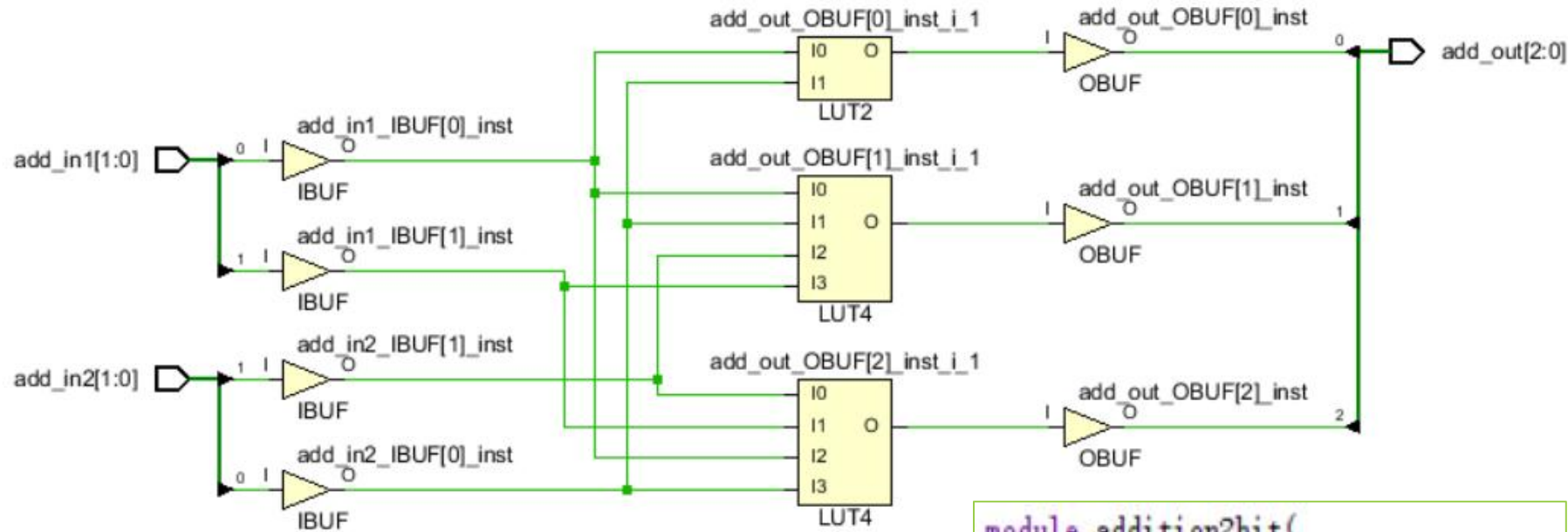


SCHEMATIC IN 'RTL ANALYSIS'

```
module lab3_df(  
    input x,  
    input y,  
    output q1,  
    output q2,  
    output q3  
);  
    assign q1 = x;  
    assign q2 = x | (x & y);  
    assign q3 = x & (x | y);  
endmodule
```



SCHEMATIC IN 'SYNTHESIS'(1)



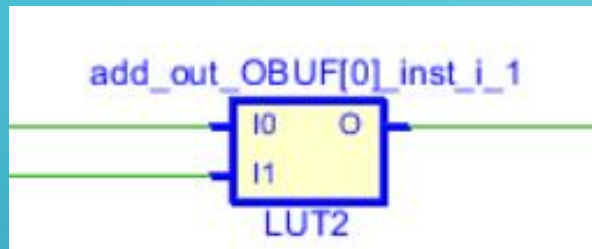
```
module addition2bit(  
    input [1:0] add_in1,  
    input [1:0] add_in2,  
    output [2:0] add_out  
);  
    assign add_out = add_in1+add_in2;  
endmodule
```

SYNTHESIS

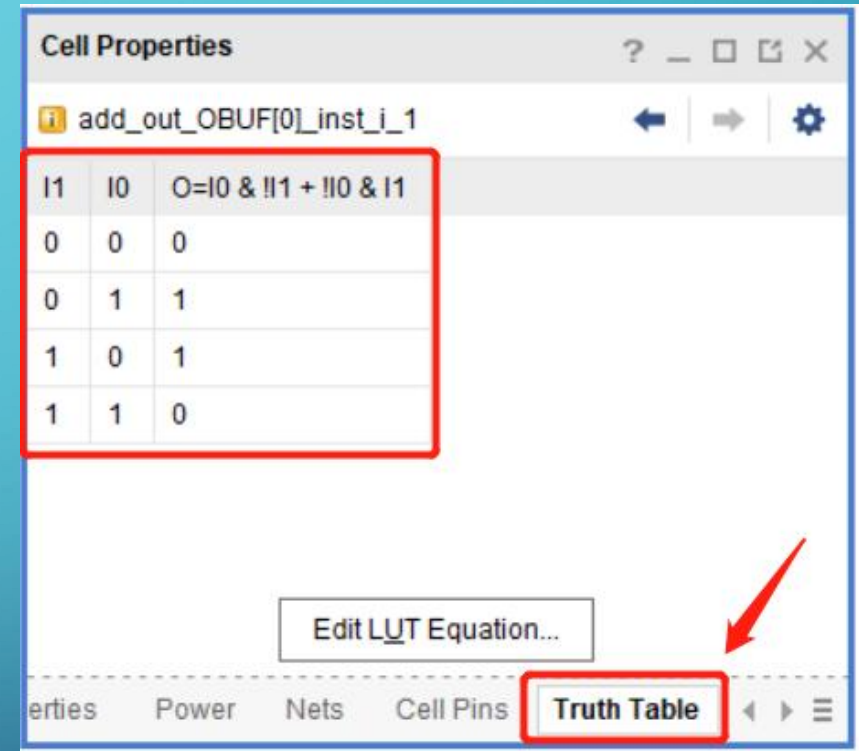
- Run Synthesis ¹
- Open Synthesized Design ²
 - Constraints Wizard
 - Edit Timing Constraints
 - Set Up Debug
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction
 - Report Methodology
 - Report DRC
 - Report Utilization
 - Report Power
 - Schematic ³

SCHEMATIC IN 'SYNTHESIS'(2)

- **Double click** the **LUT**(Look Up Table) in schematic window



- In the '**Cell Properties**' window , choose '**Truth Table**' , the truth table of the cell is shown



PRACTICE1

- 1. Do the circuit design:
 - A 3-input circuit is described in the right truth table.
 - Implement the circuit by using data flow(using assign and bit operators, in this way, set its output port name as 'Fd') and in som(Sum of Minterms) form.
- 2. Get the schematic of the circuit in “RTL analysis” and “Synthesis” respectively, describe the differences between them.
- 3. Create testbench, do simulation to verify function of the design.
 - The simulation time is 80ns, and all states of the input signal are covered during the simulation time.
 - Using system tasks or functions to display the simulation result(time, inputs and outputs) in “**Tcl Console**” sub-window of Vivado.

x	y	z	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

PRACTICE2(1)

1. rewrite the following module in 'data flow' style by verilog.

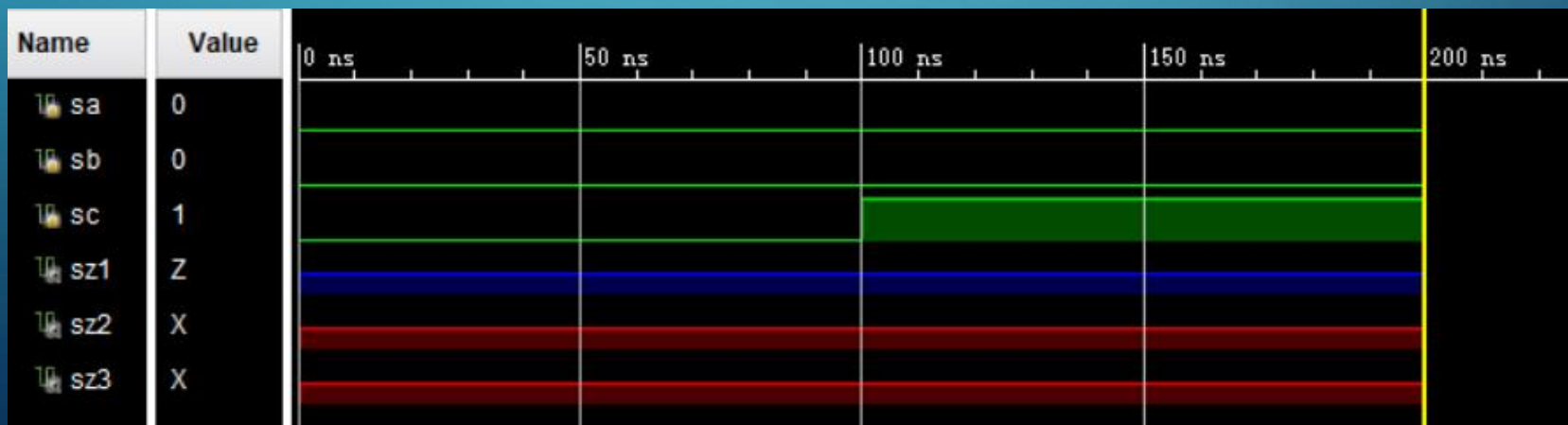
```
module sop_som_pom( input a, b, c, output z1, z2, z3 );  
  
    wire na,nb,nc;  
    not un1(na,a), un2(nb,b), un3(nc,c);  
  
    wire ua1_o;  
    and ua1( ua1_o, nb, c );  
    or uo1( z1, ua1_o, a );  
  
    wire ua2_o, ua3_o, ua4_o, ua5_o, ua6_o;  
    and ua2( ua2_o, na, nb, c ), ua3( ua3_o, a, nb, nc ), ua4( ua4_o, a, nb, c ), ua5( ua5_o, a, b, nc ), ua6( ua6_o, a, b, c );  
    or uo2( z2, ua2_o, ua3_o, ua4_o, ua5_o, ua6_o );  
  
    wire uo3_o, uo4_o, uo5_o;  
    or uo3(uo3_o, a, b, c), uo4(uo4_o, a, nb, c), uo5(uo5_o, a, nb, nc);  
    and ua7( z3, uo3_o, uo4_o, uo5_o);  
  
endmodule
```

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

PRACTICE2(2)

- 2. Do the simulation on sop_som_pom_sim (finished following the request in page 18) which is the testbench of sop_som_pom, it is found that the waveform is not in line with expectations: 1) There are just two values of sa, sb and sc in the simulation, why ? 2) The state of sz1 is Z, while the state of sz2 and sz3 is X, why?
- 3. Modify the testbench to make it workable and test the function of module sop_som_pom.
 - 1) The simulation time is 800ns, and all states of the input signal are covered during the simulation time.
 - 2) Using system tasks or functions to display the simulation result(time, inputs and outputs) in “**Tcl Console**” sub-window of Vivado.

```
1      `timescale 1ns / 1ps
2      //////////////////////////////////////
21     module sop_som_pom_sim( );
22         reg sa, sb, sc;
23         wire sz1, sz2, sz3;
24         //sop_som_pom u1( . a(sa), . b(sb), . c(sc), . z1(sz1), . z2(sz2), . z3(sz3));
25         sop_som_pom u1(sa, sb, sz1, sz2, sz3);
26         /*...*/
36     initial begin
37         {sa, sb, sc} = 3'b0;
38         repeat(7): begin
39             #100 {sa, sb, sc} = {sa, sb, sc} + 1;
40         end
41         #100 $finish();
42     end
43     /*...*/
59 endmodule
```



TIPS1.

SUM-OF-PRODUCT VS SUM-OF-MINTERMS

- $F(A,B,C)=A+B'C$
- $F(A,B,C)=AB(C'+C)+AB'(C'+C)+B'C(A'+A)=A'B'C+AB'C'+AB'C+ABC'+ABC$
 $=m1+m4+m5+m6+m7=\Sigma(1,4,5,6,7)$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Minterm	A	B	C	F
m0	0	0	0	0
m1	0	0	1	1
m2	0	1	0	0
m3	0	1	1	0
m4	1	0	0	1
m5	1	0	1	1
m6	1	1	0	1
m7	1	1	1	1

TIPS2.

SUM-OF-MINTERMS VS PRODUCT-OF-MAXTERMS

- $F(A,B,C) = A + B'C =$

$$AB(C'+C) + AB'(C'+C) + B'C(A'+A) = A'B'C + AB'C' + AB'C + ABC' + ABC$$

$$= m_1 + m_4 + m_5 + m_6 + m_7 = \Sigma(1,4,5,6,7)$$

Minterm	A	B	C	F	F'
m0	0	0	0	0	1
m1	0	0	1	1	0
m2	0	1	0	0	1
m3	0	1	1	0	1
m4	1	0	0	1	0
m5	1	0	1	1	0
m6	1	1	0	1	0
m7	1	1	1	1	0

$$F(A,B,C) = F(A,B,C)''$$

$$= (F(A,B,C))'$$

$$= (\Sigma(1,4,5,6,7)')'$$

$$= (m_0 + m_2 + m_3)'$$

$$= (A'B'C' + A'BC' + A'BC)'$$

$$= (A'B'C')' \cdot (A'BC')' \cdot (A'BC)'$$

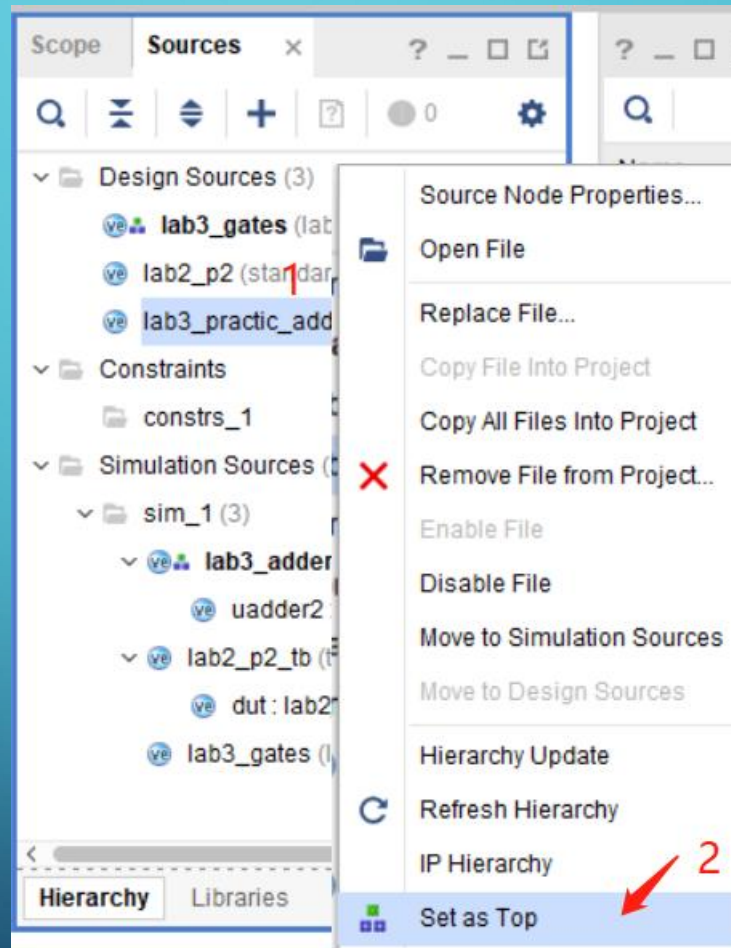
$$= (A+B+C) \cdot (A+B'+C) \cdot (A+B'+C')$$

$$= M_0 \cdot M_2 \cdot M_3 = \Pi(0,2,3)$$

VIVADO OPERATION TIPS

- In Vivado project, top module is treated as active.
 - in Design source, active module work with active constraints file to generate the bitstream file.
 - in Simulation source, active module is runned by the simulator to generate the waveform.
- There is only 1 top module in Design Sources and 1 top module in Simultion Sources.
- The top module could be set by manual.
 - 1. left click on the file to choose the one(in the demo on the right picture it is *lab3_practice_add2bit*)
 - 2. right click on it to invoke the pop-up window, click “Set as Top” in it.

Before setting, “**lab3_gates**” is top module in Design sources



After setting, the top module in Design sources has changed to be **lab3_practic_add2bit**.

